Declaration

VAAL UNIVERSITY
OF TECHNOLOGY
Inspiring thought. Shaping talent.

FACULTY:	Applied and Computer Sciences
DEPARTMENT:	Computer Sciences
SUBJECT:	Information System 1.2
SUBJECT CODE:	**AIITY1A/**
LECTURER:	**Ms.Nellylyn Moyo**
MODERATOR:	Grace Thapeli
YEAR:	2026

ASSESSMENT NAME:	Assessment 3: Project Plan
ASSESSMENT DATE:	31 October 2026

STUDENT NUMBER	SURNAME AND INITIALS	SIGNATURE
226058553	MODJADJI LRT	
226001512	MALULEKE NH	
224973142	MALULEKE H	
226048277	MATHE K	
226057509	MCHUNU N	
226067607	MAWILA TV	
222516976	TYAMZASHE O	
225259567	MHLARHI M	

Declaration

225329996	SAMBO A	
226004261	NKOSI S	

DECLARATION

We, *Lefalahae MODJADJI*

Siyabonga Nkosi

Navelo Harvey Maluleke

Oratile Tyamzashe

Hlulani Maluleke

Tamiah Vitomi Mawila

Mikateko Mhlarhi

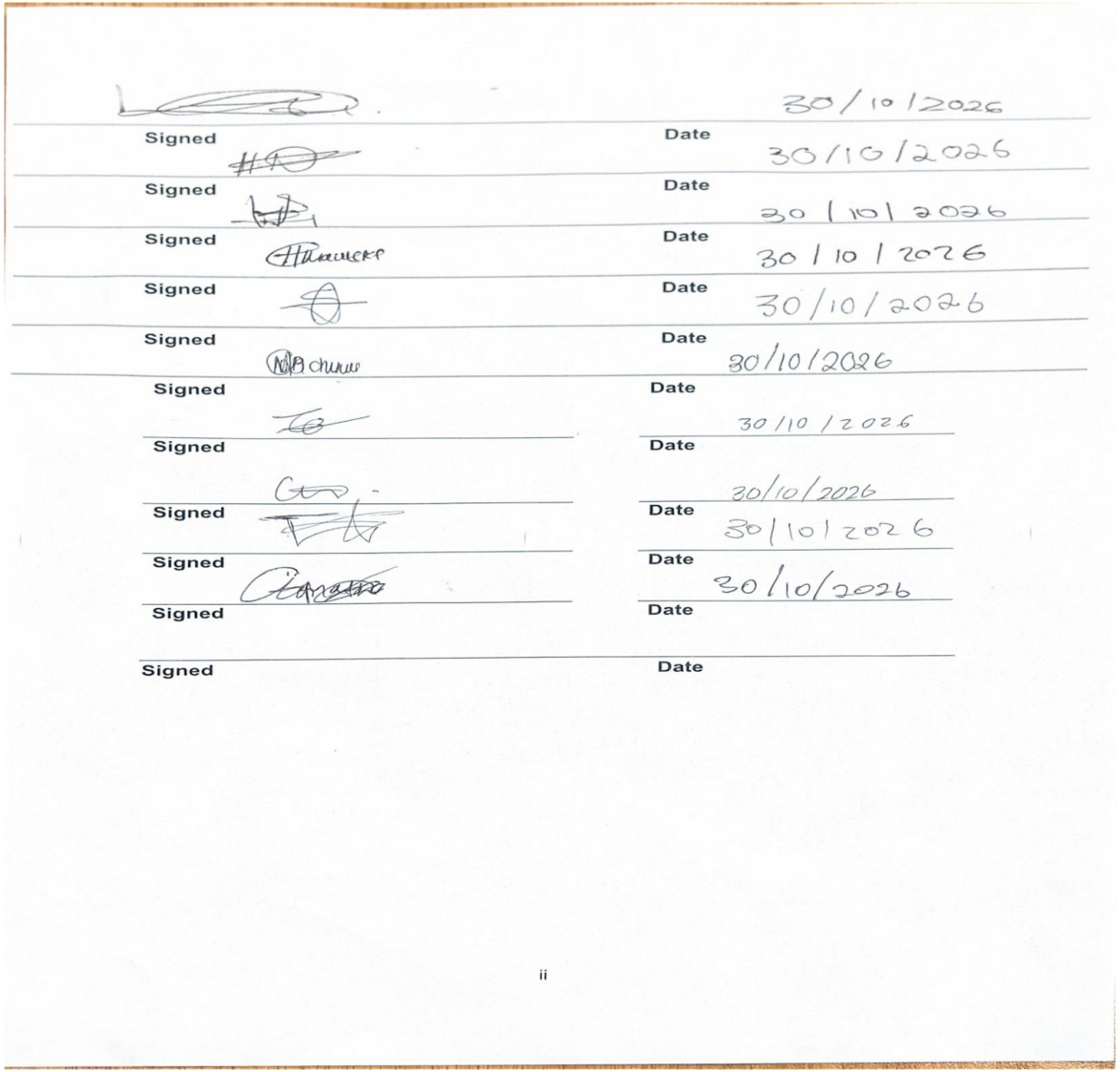
Agreement Sambo

Nolwazi Mchunu

Kgothatso Mathe

declare that the contents of this project represent our own unaided work, and that the project has not previously been submitted for academic examination towards any qualification. Furthermore, it represents our own opinions and not necessarily those of the Vaal University of Technology.

Declaration



Signed	Date
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026
	30/10/2026

G14 Hospital Management System

Group 14



The Scenario

Imagine a patient arriving at G14 Healthcare Facility in Vanderbijlpark, requiring medical attention. Before the healthcare worker can proceed, the patient's medical record must be located. However, because the facility relies on physical files and manual record-keeping finding the correct file may take longer than expected. Meanwhile, other patients continue arriving, queues become longer and staff face increasing pressure to assist everyone.

The Scenario (continued)

This situation highlights a challenge that many healthcare facilities can experience when managing large amounts of patient information using manual systems Delays in retrieving medical records can affect the patient experience and make it difficult for healthcare workers to access important information when they need it.

At G14, patient information, appointments, treatments, prescriptions and billing details are stored in physical documents and separate files. As the number of patients increases, managing these records becomes more difficult. Files may be misplaced or damaged, information may be incomplete and the same patient details may be recorded multiple times in different documents.

The Scenario (continued)

These challenges create additional work for administrative staff and may result in longer waiting times for patients. Healthcare workers may also struggle to obtain a complete view of a patient's medical history, particularly when information is spread across several files.

The main concern is that the current system makes it difficult for G14 to manage its growing information needs efficiently. Without a more organised approach, the facility may continue experiencing delays, data inconsistencies and difficulties maintaining accurate records

Introduction to the problem

G14 Healthcare Facility provides basic healthcare services to the Vanderbijlpark community. Its daily operations depend on accurate and accessible information to support patient care, appointment scheduling, treatment management, prescription records, billing and staff administration.

However, the facility's existing flat-file system is becoming less suitable as its patient population grows The system relies heavily on manual processes, making information difficult to retrieve, update and maintain consistently.

Introduction to the problem (continued)

As Group 14, we examined these challenges to understand how the current record-keeping approach affects the facility's operations. We identified several key problems that need to be addressed:

- Slow information retrieval: Staff must manually search through physical files, delaying access to patient information.
- Poor record storage: Paper documents occupy significant space and can be lost, damaged or misplaced
- Data duplication: The same patient information may be recorded in appointment, treatment, prescription and billing files, wasting storage space and increasing the risk of inconsistencies

Introduction to the problem (continued)

- Difficult information updates: Changes to patient details may need to be made across several files, creating opportunities for human error and outdated information.
- Data integrity and security concerns: Maintaining complete, accurate and protected records becomes more difficult when information is scattered across physical documents
- Inefficient reporting: Compiling reports and analysing facility operations is challenging when information is stored in separate files

Introduction to the problem (continued)

These problems affect more than the administrative side of the facility. They can contribute to patient frustration, increase staff workloads and limit the availability of information needed to support timely healthcare services

Our assessment showed that the underlying issue is the way information is organised and managed Adding more physical storage or continuing to rely on manual processes would not adequately address these limitations.

Group 14 therefore proposes a structured database solution designed around G14's information management needs. Our aim is to improve the organisation, accuracy and accessibility of records while creating a system that can support the facility as its operations expand

Proposed Solution

TO address the identified challenges, Group 14 proposes replacing G14's existing flat-file system with a relational database management system. This solution will organise the facility's information into structured, related tables rather than keeping records scattered across separate physical files

The proposed database will include tables for patients, doctors, appointments, treatments, prescriptions, bills and staff members. Each table will store information relevant to its purpose, while primary keys and foreign keys will establish relationships between the different types of records.

For example, a patients information can be linked to their appointments, treatments and prescriptions. This will make it easier for authorised staff to retrieve related Information without unnecessarily duplicating the patient's details in every record.

Proposed Solution(continued)

We will also apply database normalisation, progressing from Unnormalized Form (IJNF) through First Normal Form (INF) and Second Normal Form (2NF) to Third Normal Form (3NF). This process will help reduce unnecessary data duplication, improve consistency and maintain data integrity. In addition, appropriate access controls, security measures and backup procedures will be necessary to protect sensitive patient information. These measures will help ensure that information is accessible to authorised users while reducing the risks associated with loss or unauthorised access.

Proposed Solution(continued)

The expected outcome is a more efficient and reliable information management system that reduces the administrative burden, improves record organisation and supports better access to patient information. Although the database cannot independently guarantee shorter waiting times or better clinical outcomes, it can address important information management problems that contribute to operational delays.

In conclusion, Group 14's proposal focuses on solving the root cause Of G14’s record management challenges. By introducing a structured relational database, we aim to help the facility move away from inefficient manual processes towards a more organised, accurate and scalable system. This will provide a stronger foundation for managing growing patient numbers and supporting the delivery of reliable healthcare services to the community.

Business Rules

These business rules were acquired by interview managerial individuals of the facility, and consulting with targeted users as to improvements that would be appropriate

The following business rules describe how G14 Healthcare Facility operates and how data is managed within the proposed relational database system. These rules are designed to support day-to-day hospital operations

Business Rules(continued)

Patient Rules

- BR1: Each patient should be identified by a PatientID.
- BR2: A single patient could have one or more appointments with health care providers. BR3: Each appointment should have exactly one patient.
- BR4: One person can receive health care services from one or more billing records.
- BR5: A patient is allowed to be in the system without an appointment.

Business Rules(continued)

Staff Rules

- BR6: Staff should all have their own StaffID.
- BR7: All staff members should have a StaffType that specifies their role in the healthcare facility.
- BR8: StaffType can be any of the following values:
- Doctor
- Nurse
- Receptionist
- Administrator
- BR9: Staff person can be in multiple treatments.
- BR10: Doctors and Nurses only can be assigned to treatments.
- BR11: Receptionists and Administrators are not allowed to give treatments or to supervise them.

Business Rules(continued)

Appointment Rules

- BR12: An AppointmentID is required to be unique for each appointment.
- BR13: Each appointment is supposed to have one patient assigned to it.
- BR14: Multiple dates and times can be planned for a patient.
- BR15: Patient must be in system for an appointment to be created.
- BR16: An appointment may lead to treatment without a hospital admission.
- BR17: An appointment can yield 0 or 1 admissions.
- BR18: A zero, one, or many prescriptions are possible to receive during an appointment.
- BR19: The status of the appointment should be reflected in the appointment status.

Business Rules(continued)

Admission Rules

- BR20: Must ensure that each admission has a unique AdmissionID.
- BR21: All admissions should be from one appointment.
- BR22: Each admission must be given to a room.
- BR23: Not all appointments result in admissions.
- BR24: No admission can be made without the existence of an appointment and room assignment.
- BR25: Admission and discharge dates should be documented to be able to monitor patient's stay.
- BR26: It is possible for a patient to have several admissions over time.
-

Business Rules(continued)

Ward Rules

- BR27: Every ward should have a WardID.
- BR28: A ward can have one or more rooms.
- BR29: Each room should only be assigned to one ward.
- BR30: A ward can have a number of rooms before being allocated.

Business Rules(continued)

Room Rules

- BR31: All rooms need to be assigned a RoomID.
- BR32: All rooms will need to be designated to a ward.
- BR33: A room can be opened for several different admissions over time.
- BR34: One patient can only be admitted to a room at a given time.
- BR35: All rooms are required to have a RoomAvailability status.
-
- BR36: RoomAvailability can have one of the following values:
- Available
- Occupied
- BR37: A patient can only be put in a room that has the status "Available".
- BR38: When the patient is admitted to the room, the status of the room should be set to "Occupied".
- BR39: The room should be set to "Available" when the patient is discharged from the room.

Business Rules(continued)

Treatment Rules

- BR40: Treatments must have a unique TreatmentID.
- BR41: All treatments should have an appointment.
- BR42: All treatments are to be carried out or supervised by one member of staff.
- BR43: Depending on the patient, one or more treatments may be required for an appointment.
- BR44: Record treatment details such as diagnosis and treatment description for all treatment given.
- BR45: If there is no valid appointment, then there is no treatment.
- BR46: A treatment may be provided to either an outpatient or an admitted patient.

Business Rules(continued)

Medicine Rules

- BR47: A MedicineID should be unique for each medicine.
- BR48: There must exist a MedicineName for every medicine.
- BR49: Stock value should be assigned to every medicine to show the amount of medicine available.
- BR50: A drug can be prescribed for several different prescriptions.
- BR51: A medicine can be in the system without a prescription.
- BR52: Stock levels should be adjusted whenever medication is supplied via a prescription.

Business Rules(continued)

Prescription Rules

- BR53: A PrescriptionID must be unique for each prescription.
- BR54: Each prescription should be linked to an appointment.
- BR55: All prescriptions include one medicine.
- BR56: There must be an appointment when a prescription is issued.
- BR57: For every prescription there is a medicine that goes with it.
- BR58: Multiple prescriptions can be issued for one appointment.
- BR59: A Medicine can be issued on more than one Prescription.
- BR60: There shall be records of dosage for each prescription.

Business Rules(continued)

Billing Rules

- BR61: Each BillingRecord must be distinctively represented by a BillID.
- BR62: Each bill must have the identifier of one patient.
- BR63: A patient may have multiple billing records.
- BR64: PaymentStatus is required to have one of the following values:
- Paid
- Pending
- Overdue
- BR65: Billing records shall be accurate to reflect services provided to the patient during the course of appointments, admissions, treatments and prescriptions.

Business Rules(continued)

Relationship and Cardinality Summary

- One Patient can have many Appointments.
- One Patient can have many Billing records.
- One Appointment can have many Treatments.
- One Appointment can have many Prescriptions.
- Zero or one Admission can be created from one Appointment.
- There may be several Rooms in each Ward.
- One Room can accommodate many Admissions over time.
- Many Treatments can be done by one Staff member.
- One Medicine can be found in several Prescriptions.

Business Rules (Summary)

These business rules help to ensure that the proposed relational database will meet the business requirements of G14 Healthcare Facility. The rules define clearly the relationship between patients, appointments, admissions, wards, rooms, staff, treatments, medicines, prescriptions and billing records, while also guaranteeing data consistency, avoiding data duplication through Third Normal Form (3NF), and supporting both outpatient and inpatient healthcare services.

Database Normalization

Database normalization was done to convert the existing flat file system of the G14 Healthcare Facility into a structured relational database. This current flat-file system combines patient, staff, treatment, room and billing data all in the same file, which leads to duplicated records, inefficient data retrieval and challenges with data updates.

The normalization process was performed from First Normal Form (1NF) to Third Normal Form (3NF) in order to reduce data duplication, ensure data integrity and to design the database structure to meet the requirements of G14 Healthcare Facility's operations.

Unnormalized_Form (UNF)

In the original flat-file system used by G14 Healthcare Facility, information relating to patients, appointments, admissions, rooms, wards, staff members, treatments, medicines, prescriptions and billing records was stored together in a single table.

UNF (continued) unnormarlized Data

PatientID	PatientName	AppointmentID	TreatmentID	PrescriptionID	MedicineName	WardName	BillAmount
P001	John Smith	A001	T001	PR001	Amlodipine	General	1500
P001	John Smith	A001	T002	PR002	Aspirin	General	1500
P002	Mary Jones	A002	T003	PR003	Paracetamol	Surgical	2200
P003	David Brown	A003	T004	PR004	Metformin	ICU	1800

UNF (continued)

Problems Identified

- Patient information is repeated.
- Appointment information is repeated.
- Medicine information is repeated.
- Billing information appears multiple times.
- Data redundancy exists throughout the table.
- Updating, deleting, or inserting data may result in inconsistent data(anomalies).

First Normal Form (1NF)

Examples:

Patient

PatientID	FirstName	LastName	PhoneNumber
P001	John	Smith	0823456789
P002	Mary	Jones	0834567890
P003	David	Brown	0845678901

First Normal Form (1NF)examples

Appointment

AppointmentID	PatientID	AppointmentDate	AppointmentTime	AppointmentStatus
A001	P001	01/04/2026	08:00	Completed
A002	P002	03/04/2026	09:30	Completed
A003	P003	05/04/2026	10:00	Completed

First Normal Form (1NF) examples

Admission

AdmissionID	AppointmentID	PatientID	RoomID	AdmissionDate	DischargeDate
AD001	A001	P001	R01	01/04/2026	05/04/2026
AD002	A002	P002	R02	03/04/2026	07/04/2026

First Normal Form (1NF) examples

Treatment

TreatmentID	AppointmentID	StaffID	Diagnosis
T001	A001	S001	Hypertension
T002	A001	S002	Monitoring
T003	A002	S003	Influenza

First Normal Form (1NF) examples

Medicine

MedicineID	MedicineName	Stock
M001	Amlodipine	150
M002	Aspirin	300
M003	Paracetamol	500

First Normal Form (1NF) examples

Prescription

PrescriptionID	AppointmentID	MedicineID	Dosage
PR001	A001	M001	5mg Daily
PR002	A001	M002	100mg Daily
PR003	A002	M003	500mg Every 6 Hours

1NF Outcomes

After going through 1NF there following improvements are noted:

- Repeating groups were eliminated.
- Atomic values were enforced.
- Each row became uniquely identifiable.
- Data became easier to manage.

Second Normal Form(2NF)

To achieve Second Normal Form (2NF), all non-key attributes are made to be dependent the primary keys.

Examples of the tables in 2NF:

Staff

StaffID	StaffType	Initials	PhoneNumber
S001	Doctor	JM	1236547890
S002	Nurse	PN	0174852369
S003	Doctor	LK	0321458796

2NF (Examples)

Ward

WardID	WardName
W01	General
W02	Surgical
W03	ICU

2NF (cont)

Billing

BillID	PatientID	Amount	PaymentStatus
B001	P001	1500	Paid
B002	P002	2200	Pending
B003	P003	1800	Paid

2NF (cont)

Room

RoomID	WardID	WardName	RoomNumber	RoomAvailability
R01	W01	General	101	Occupied
R02	W02	Pediatrics	201	Occupied
R03	W01	General	102	Occupied
R04	W03	Surgical	301	Occupied
R05	W02	Pediatrics	202	Occupied

2NF (cont)

After the 2NF stage, non-key attributes are depended on the primary keys of they’re entries and the following improvements can be seen:

- Partial dependencies were removed.
- Data was separated into subject-specific entities.
- Medicine information was no longer duplicated.
- Prescription management improved.
- Data redundancy was significantly reduced.

But a problem still exists on the Room table; Although data duplication has largely been removed, WardName is still dependent on WardID rather than RoomID, creating a transitive dependency, thus violating 3NF.this takes us to the next step 3NF.

Third Normal Form (3NF)

To achieve Third Normal Form (3NF), all transitive dependencie are removed. That leaves us with the following table structure:

Room

The transistive dependency is removed, thus achieving 3NF.

RoomID	WardID	RoomNumber	RoomAvailability
R01	W01	101	Occupied
R02	W02	201	Occupied
R03	W01	102	Occupied
R04	W03	301	Occupied
R05	W02	202	Occupied

Final 3NF Relations

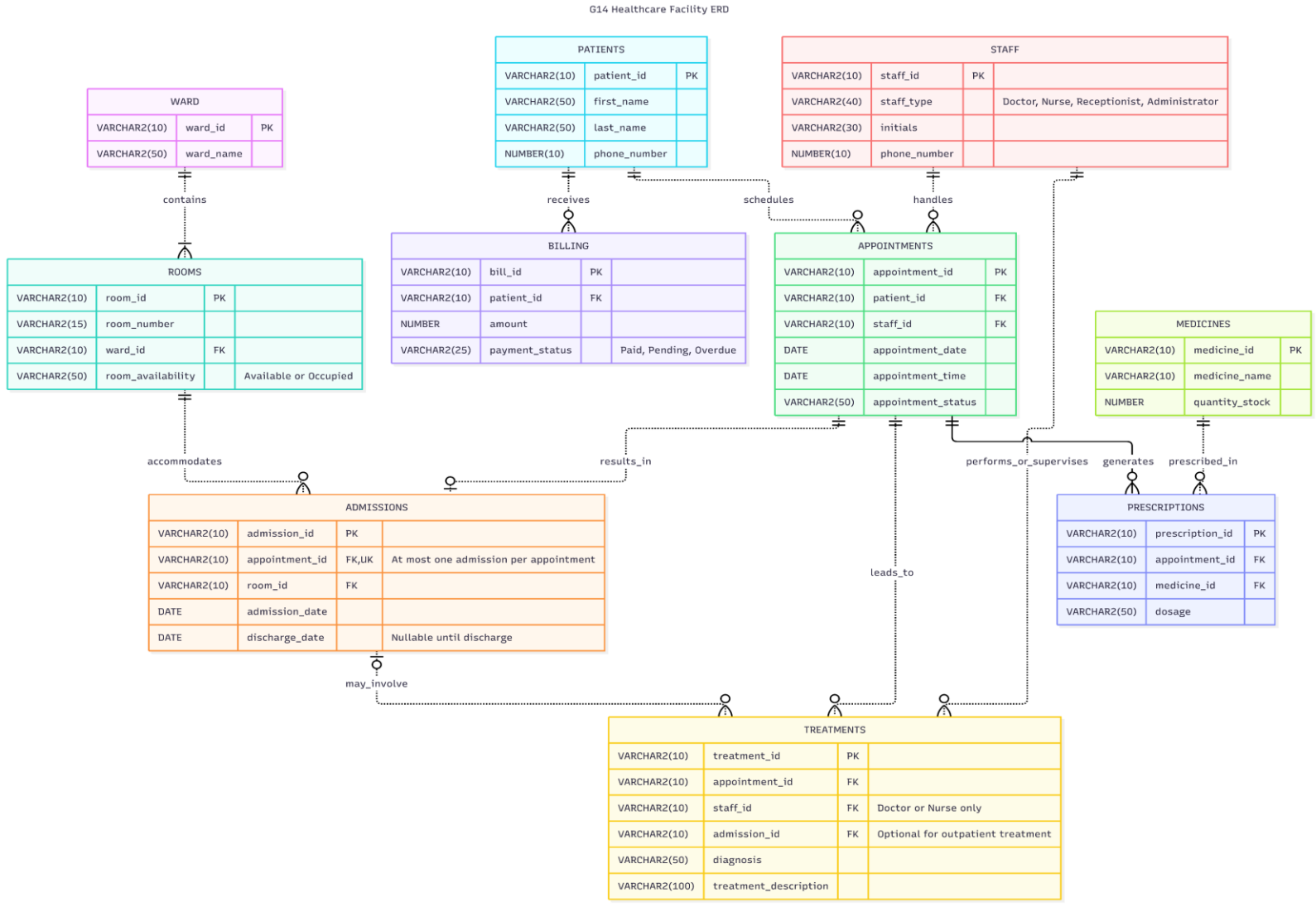
Entities	Attributes
Patient	PatientID (PK), FirstName, LastName, PhoneNumber
Appointment	AppointmentID (PK), PatientID (FK), AppointmentDate, AppointmentTime, AppointmentStatus
Admission	AdmissionID (PK), AppointmentID (FK), PatientID (FK), RoomID (FK), AdmissionDate, DischargeDate
Ward	WardID (PK), WardName
Room	RoomID (PK), WardID (FK), RoomNumber, RoomAvailability
Staff	StaffID (PK), StaffType, Initials
Treatment	TreatmentID (PK), AppointmentID (FK), StaffID (FK), Diagnosis, TreatmentDescription
Medicine	MedicineID (PK), MedicineName, Stock
Prescription	PrescriptionID (PK), AppointmentID (FK), MedicineID (FK), Dosage
Billing	BillID (PK), PatientID (FK), Amount, PaymentStatus

3NF (cont): Outcomes

We achieve the following because of 3NF:

- Repeating groups are eliminated.
- Partial dependencies are removed.
- Transitive dependencies are removed.
- Medicine data is separated from prescription data.
- Appointment information was stored once.
- Foreign keys link related tables.
- Data integrity is improved.
- Redundancy was minimized throughout the database.

The Resulting ERD

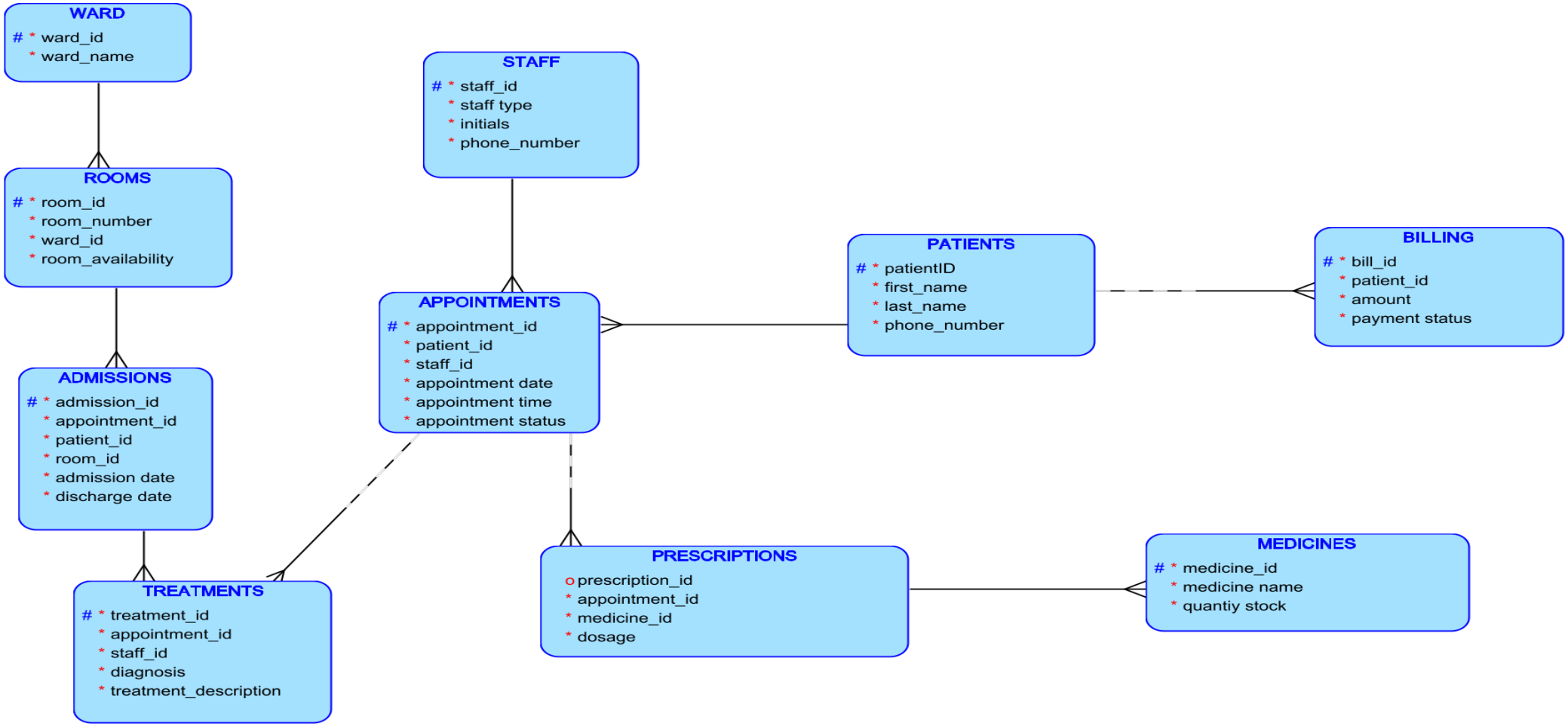


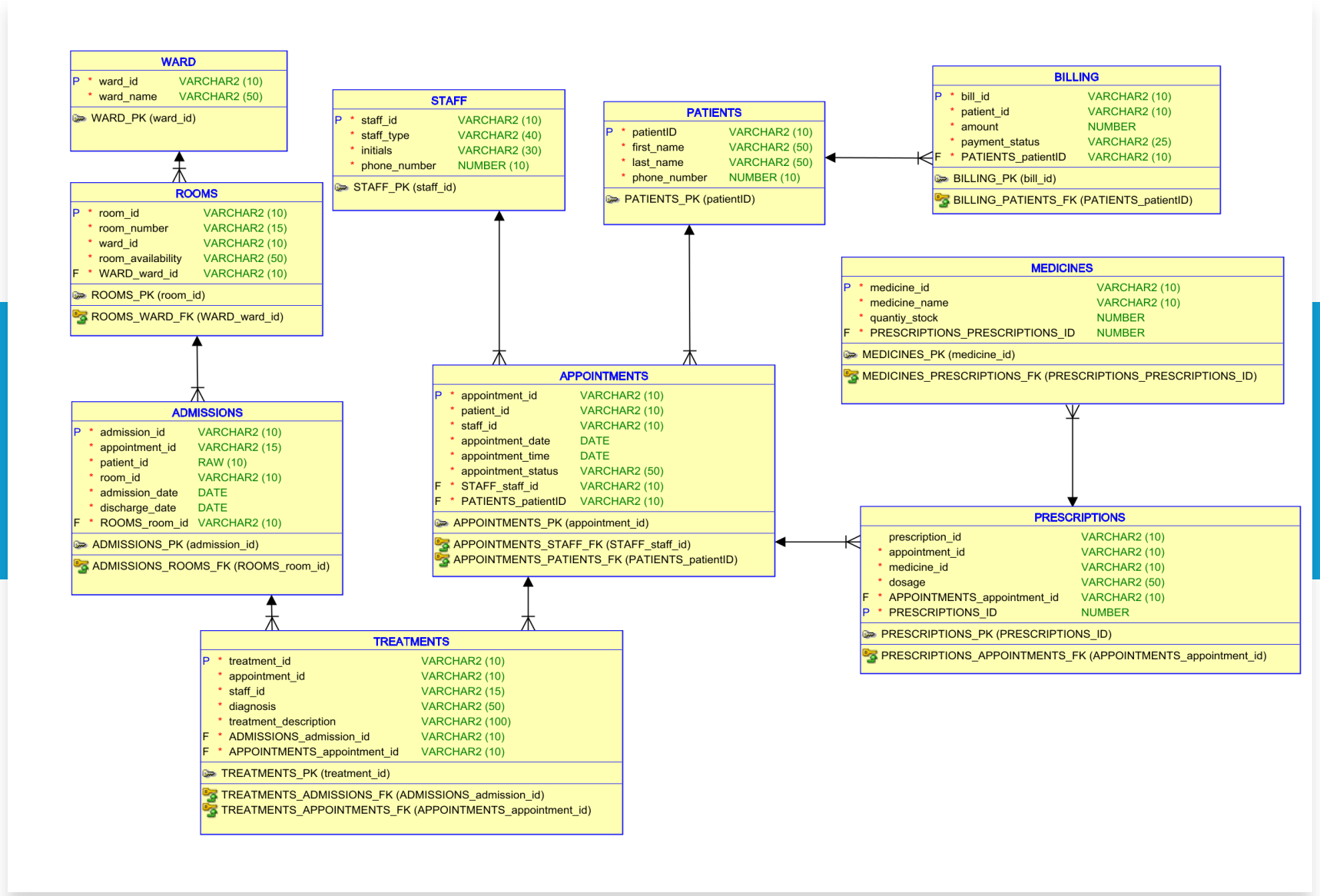
Normalization Summary

The G14 Healthcare Facility database was normalized from Unnormalized Form (UNF) through First Normal Form (1NF), Second Normal Form (2NF), and finally Third Normal Form (3NF). The normalization process eliminated redundant data, improved consistency, and ensured efficient data storage. The final database structure provides an integrated solution for managing patients, appointments, admissions, wards, rooms, staff members, treatments, medicines, prescriptions, and billing records while supporting both outpatient and inpatient healthcare services.

With the database of G14 normalized we can move to the implementation or testing of the physical database.

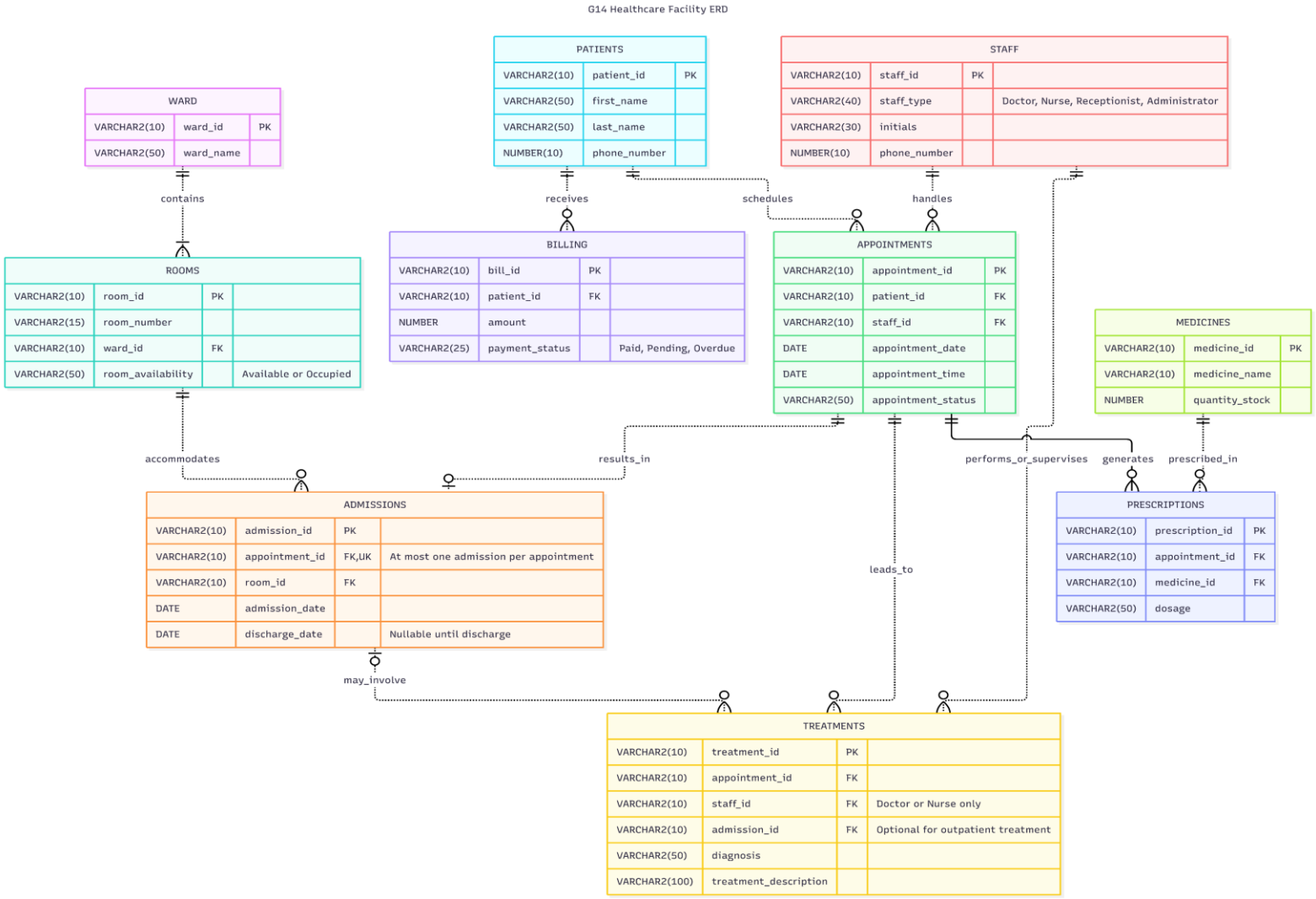
ERD (Planning) :Logical





ERD (Relational)

ERD (with crows foot notation)



SQL (Creating Tables)

```
-- PATIENTS
-- ========================================
CREATE TABLE PATIENTS (
    patient_id      VARCHAR(10) NOT NULL,
    first_name      VARCHAR(50) NOT NULL,
    last_name       VARCHAR(50) NOT NULL,
    phone_number    VARCHAR(10) NOT NULL,
    CONSTRAINT PATIENTS_PK
        PRIMARY KEY (patient_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- ========================================
-- STAFF
-- ========================================
CREATE TABLE STAFF (
   staff_id       VARCHAR(10) NOT NULL,
   staff_type     VARCHAR(40) NOT NULL,
   initials       VARCHAR(30) NOT NULL,
   phone_number   VARCHAR(10),
   CONSTRAINT STAFF_PK
      PRIMARY KEY (staff_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- WARD
-- ========================================
CREATE TABLE WARD (
   ward_id        VARCHAR(10) NOT NULL,
   ward_name      VARCHAR(50) NOT NULL,
   CONSTRAINT WARD_PK
      PRIMARY KEY (ward_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- ROOMS
-- ====================================
CREATE TABLE ROOMS (
    room_id VARCHAR(10) NOT NULL,
    room_number VARCHAR(15) NOT NULL,
    ward_id VARCHAR(10) NOT NULL,
    room_availability VARCHAR(50) NOT NULL,
```

SQL (Creating Tables)rooms cont

```
  CONSTRAINT ROOMS_PK
    PRIMARY KEY (room_id),
  CONSTRAINT ROOMS_WARD_FK
    FOREIGN KEY (ward_id)
    REFERENCES WARD (ward_id),
  CONSTRAINT ROOM_STATUS_CHK
    CHECK (
      room_availability IN (
        'Available',
        'Occupied'
      )
    )
);
-- ========================================
```

SQL (Creating Tables

```
-- APPOINTMENTS
-- ========================================
CREATE TABLE APPOINTMENTS (
   appointment_id     VARCHAR(10) NOT NULL,
   patient_id         VARCHAR(10) NOT NULL,
   appointment_date    DATE NOT NULL,
   appointment_time    VARCHAR(20) NOT NULL,
   appointment_status  VARCHAR(50) NOT NULL,
   CONSTRAINT APPOINTMENTS_PK
      PRIMARY KEY (appointment_id),
   CONSTRAINT APPOINTMENTS_PATIENT_FK
      FOREIGN KEY (patient_id)
      REFERENCES PATIENTS (patient_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- ========================================
-- ADMISSIONS
-- ========================================
CREATE TABLE ADMISSIONS (
   admission_id       VARCHAR(10) NOT NULL,
   appointment_id     VARCHAR(10) NOT NULL,
   patient_id         VARCHAR(10) NOT NULL,
   room_id            VARCHAR(10) NOT NULL,
   admission_date     DATE NOT NULL,
   discharge_date     DATE NOT NULL,
   CONSTRAINT ADMISSIONS_PK
      PRIMARY KEY (admission_id),
```

SQL (Creating Tables)admissions cont

```
  CONSTRAINT ADMISSIONS_APPOINTMENT_FK
    FOREIGN KEY (appointment_id)
    REFERENCES APPOINTMENTS (appointment_id),
  CONSTRAINT ADMISSIONS_PATIENT_FK
    FOREIGN KEY (patient_id)
    REFERENCES PATIENTS (patient_id),
  CONSTRAINT ADMISSIONS_ROOM_FK
    FOREIGN KEY (room_id)
    REFERENCES ROOMS (room_id)
);
-- ============================================
```

SQL (Creating Tables

```
-- TREATMENTS
-- ========================================
CREATE TABLE TREATMENTS (
    treatment_id          VARCHAR(10) NOT NULL,
    appointment_id        VARCHAR(10) NOT NULL,
    staff_id              VARCHAR(10) NOT NULL,
    diagnosis             VARCHAR(100) NOT NULL,
    treatment_description  VARCHAR(200) NOT NULL,
```

SQL (Creating Tables)treatments cont

```
CONSTRAINT TREATMENTS_PK
      PRIMARY KEY (treatment_id),
   CONSTRAINT TREATMENTS_APPOINTMENT_FK
      FOREIGN KEY (appointment_id)
      REFERENCES APPOINTMENTS (appointment_id),
   CONSTRAINT TREATMENTS_STAFF_FK
      FOREIGN KEY (staff_id)
      REFERENCES STAFF (staff_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- MEDICINES
-- ========================================
CREATE TABLE MEDICINES (
    medicine_id     VARCHAR(10) NOT NULL,
    medicine_name   VARCHAR(50) NOT NULL,
    stock           NUMBER NOT NULL,
    CONSTRAINT MEDICINES_PK
        PRIMARY KEY (medicine_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- PRESCRIPTIONS
-- ================================================
CREATE TABLE PRESCRIPTIONS (
    prescription_id     VARCHAR(10) NOT NULL,
    appointment_id      VARCHAR(10) NOT NULL,
    medicine_id         VARCHAR(10) NOT NULL,
    dosage              VARCHAR(50) NOT NULL,
```

SQL (Creating Tables)prescription cont

```
    CONSTRAINT PRESCRIPTIONS_PK
        PRIMARY KEY (prescription_id),
    CONSTRAINT PRESCRIPTIONS_APPOINTMENT_FK
        FOREIGN KEY (appointment_id)
        REFERENCES APPOINTMENTS (appointment_id),
    CONSTRAINT PRESCRIPTIONS_MEDICINE_FK
        FOREIGN KEY (medicine_id)
        REFERENCES MEDICINES (medicine_id)
);
-- ========================================
```

SQL (Creating Tables

```
-- BILLING
-- ========================================
CREATE TABLE BILLING (
   bill_id          VARCHAR(10) NOT NULL,
   patient_id       VARCHAR(10) NOT NULL,
   amount           NUMBER(10,2) NOT NULL,
   payment_status    VARCHAR(25) NOT NULL,
   CONSTRAINT BILLING_PK
      PRIMARY KEY (bill_id),
   CONSTRAINT BILLING_PATIENT_FK
      FOREIGN KEY (patient_id)
      REFERENCES PATIENTS (patient_id)
);
-- ========================================
```

SQL (Check Constraints)

```
-- CHECK CONSTRAINTS
-- ========================================
ALTER TABLE STAFF
ADD CONSTRAINT STAFF_TYPE_CHK
CHECK (
   staff_type IN
   (
      'Doctor',
      'Nurse',
      'Receptionist',
      'Administrator'
   )
);
```

SQL (Check Constraints)

```
ALTER TABLE BILLING
ADD CONSTRAINT PAYMENT_STATUS_CHK
CHECK (
   payment_status IN
   (
      'Paid',
      'Pending',
      'Overdue'
   )
);
-- ========================================
-- END OF SCRIPT
-- ========================================
```

SQL (Insert Values into Tables)

PATIENTS

```
INSERT INTO PATIENTS
VALUES ('P001','JOHN','DOE','0720017896');
INSERT INTO PATIENTS
VALUES ('P002','JANE','DOE','0620017894');
INSERT INTO PATIENTS
VALUES ('P003','JOSE','FLICK','0745236987');
INSERT INTO PATIENTS
VALUES ('P004','LILY','SMITH','0147852369');
```

SQL (Insert Values into Tables)

```
INSERT INTO PATIENTS
VALUES ('P005','WILL','FOSTER','3216547899');
INSERT INTO PATIENTS
VALUES ('P006','KATLEGO','DUBE','369963258');
INSERT INTO PATIENTS
VALUES ('P007','MINKATEKO','MAHLANGU','1478523690');
INSERT INTO PATIENTS
VALUES ('P008','KATE','NDLOVU','2587419630');
INSERT INTO PATIENTS
VALUES ('P009','BIG','JOHN','3217896540');
```

SQL (Insert Values into Tables)

STAFF

```
INSERT INTO STAFF
VALUES ('SD001', 'Doctor', 'DR N NAIDOO', '1478523690');
INSERT INTO STAFF
VALUES ('SN001 ', 'Nurse', 'L.MAHLANGU ', '0789825530');
INSERT INTO STAFF
VALUES ('SN002 ', 'Nurse', 'MN.SMITH ', '0780789681');
INSERT INTO STAFF
VALUES (' SN003', 'Nurse', 'G.OLESENG ', '0748596321');
```

SQL (Insert Values into Tables)

```
INSERT INTO STAFF
VALUES (' SD002', 'Doctor', 'DR.L.SMITH ', '0147741025');
INSERT INTO STAFF
VALUES (' SD003', 'Dcotor', 'DR J.DOE ', '3698520147');
INSERT INTO STAFF
VALUES (' SA001', 'Administrator', 'N.MOKOENA ', '3698520147');
INSERT INTO STAFF
VALUES (' SA002', 'Administrator', 'K.NAUME ', '3217896691');
INSERT INTO STAFF
VALUES (' SR001', 'Receptionist', 'L.HOLLY ', '9632587410');
```

SQL (Insert Values into Tables)

WARD

```
INSERT INTO WARD VALUES ('W01', 'General');
INSERT INTO WARD VALUES ('W02', 'Surgical');
INSERT INTO WARD VALUES ('W03', 'ICU');
```

SQL (Insert Values into Tables)

ROOMS

```
INSERT INTO ROOMS VALUES ('R01', '101', 'W01', 'Occupied');
INSERT INTO ROOMS VALUES ('R02', '201', 'W02', 'Occupied');
INSERT INTO ROOMS VALUES ('R03', '102', 'W01', 'Available');
INSERT INTO ROOMS VALUES ('R04', '301', 'W03', 'Occupied');
INSERT INTO ROOMS VALUES ('R05', '202', 'W02', 'Available');
```

SQL (Insert Values into Tables)

APPOINTMENTS

```
INSERT INTO APPOINTMENTS
VALUES ('A001','P001',TO_DATE('01-04-2026','DD-MM-YYYY'),'08:00','Completed');
INSERT INTO APPOINTMENTS
VALUES ('A002','P002',TO_DATE('03-04-2026','DD-MM-YYYY'),'09:30','Completed');
INSERT INTO APPOINTMENTS
VALUES ('A003','P003',TO_DATE('05-04-2026','DD-MM-YYYY'),'10:00','Completed');
INSERT INTO APPOINTMENTS
VALUES ('A004','P004',TO_DATE('08-04-2026','DD-MM-YYYY'),'11:00','Scheduled');
```

SQL (Insert Values into Tables)

```
INSERT INTO APPOINTMENTS
VALUES ('A005','P005',TO_DATE('09-04-2026','DD-MM-YYYY'),'13:00','Completed');
INSERT INTO APPOINTMENTS
VALUES ('A006','P006',TO_DATE('10-04-2026','DD-MM-YYYY'),'14:30','Scheduled');
INSERT INTO APPOINTMENTS
VALUES ('A007','P007',TO_DATE('11-04-2026','DD-MM-YYYY'),'15:00','Cancelled');
INSERT INTO APPOINTMENTS
VALUES ('A008','P008',TO_DATE('12-04-2026','DD-MM-YYYY'),'09:00','Completed');
```

SQL (Insert Values into Tables)

ADMISSIONS

(note that not all appointments lead to admission)

```
INSERT INTO ADMISSIONS
VALUES ('AD001','A001','P001','R01',
      TO_DATE('01-04-2026','DD-MM-YYYY'),
      TO_DATE('05-04-2026','DD-MM-YYYY'));
INSERT INTO ADMISSIONS
VALUES ('AD002','A002','P002','R02',
```

SQL (Insert Values into Tables)

```
        TO_DATE('03-04-2026','DD-MM-YYYY'),
        TO_DATE('07-04-2026','DD-MM-YYYY'));
INSERT INTO ADMISSIONS
VALUES ('AD003','A005','P005','R04',
        TO_DATE('09-04-2026','DD-MM-YYYY'),
        TO_DATE('12-04-2026','DD-MM-YYYY'));
```

SQL (Insert Values into Tables)

TREATMENTS

```
INSERT INTO TREATMENTS
VALUES ('T001','A001','SD001',
    'Hypertension',
    'Blood pressure treatment and monitoring');
INSERT INTO TREATMENTS
VALUES ('T002','A001','SN001',
    'Observation',
    'Patient vital signs monitored');
INSERT INTO TREATMENTS
VALUES ('T003','A002','SD002',
    'Influenza',
    'Antiviral treatment prescribed');
```

SQL (Insert Values into Tables)

```
INSERT INTO TREATMENTS
VALUES ('T004','A003','SD001',
    'Diabetes',
    'Blood glucose assessment');
INSERT INTO TREATMENTS
VALUES ('T005','A005','SN002',
    'Post-operative Care',
    'Routine monitoring after surgery');
INSERT INTO TREATMENTS
VALUES ('T006','A008','SD002',
    'Respiratory Infection',
```

SQL (Insert Values into Tables)

MEDICINES

```
INSERT INTO MEDICINES
VALUES ('M001','Amlodipine',150);
INSERT INTO MEDICINES
VALUES ('M002','Aspirin',300);
INSERT INTO MEDICINES
VALUES ('M003','Paracetamol',500);
INSERT INTO MEDICINES
VALUES ('M004','Metformin',250);
```

SQL (Insert Values into Tables)

```
INSERT INTO MEDICINES
VALUES ('M005','Amoxicillin',200);
        INSERT INTO MEDICINES
VALUES ('M006','Panado',50);
INSERT INTO MEDICINES
VALUES ('M007,'Grandpa',10);
INSERT INTO MEDICINES
VALUES ('M008','Benelin',80);
```

SQL (Insert Values into Tables)

PRESCRIPTIONS

```
INSERT INTO PRESCRIPTIONS
VALUES ('PR001','A001','M001','5mg Daily');
INSERT INTO PRESCRIPTIONS
VALUES ('PR002','A001','M002','100mg Daily');
INSERT INTO PRESCRIPTIONS
VALUES ('PR003','A002','M003','500mg Every 6 Hours');
INSERT INTO PRESCRIPTIONS
VALUES ('PR004','A003','M004','500mg Twice Daily');
INSERT INTO PRESCRIPTIONS
VALUES ('PR005','A005','M002','75mg Daily');
INSERT INTO PRESCRIPTIONS
VALUES ('PR006','A008','M005','500mg Three Times Daily');
```

SQL (Insert Values into Tables)

BILLING

```
INSERT INTO BILLING
VALUES ('B001','P001',1500.00,'Paid');
INSERT INTO BILLING
VALUES ('B002','P002',2200.00,'Pending');
INSERT INTO BILLING
VALUES ('B003','P003',1800.00,'Paid');
INSERT INTO BILLING
VALUES ('B004','P004',750.00,'Pending');
```

SQL (Insert Values into Tables)

```
INSERT INTO BILLING
VALUES ('B005','P005',3200.00,'Paid');
INSERT INTO BILLING
VALUES ('B006','P006',600.00,'Pending');
INSERT INTO BILLING
VALUES ('B007','P008',1250.00,'Overdue');
INSERT INTO BILLING
VALUES ('B008','P009',950.00,'Pending');
```

SQL (Query Results From Oracle LiveSql) (screenshot)

PATIENTS

	PATIENT_ID	FIRST_NAME	LAST_NAME	PHONE_NUMBER
1	P001	JOHN	DOE	0720017896
2	P002	JANE	DOE	0620017894
3	P003	JOSE	FLICK	0745236987
4	P004	LILY	SMITH	0147852369
5	P005	WILL	FOSTER	3216547899
6	P006	KATLEGO	DUBE	369963258
7	P007	MINKATEKO	MAHLANGU	1478523690
8	P008	KATE	NDLOVU	2587419630
9	P009	BIG	JOHN	3217896540

SQL (Query Results From Oracle LiveSql) (screenshot)

STAFF

Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download ▾ Execution time: 0.007 seconds

	STAFF_ID	STAFF_TYPE	INITIALS	PHONE_NUMBER
1	SD001	Doctor	DR N NAIDOO	1478523690
2	SN001	Nurse	L.MAHLANGU	0789825530
3	SN002	Nurse	MN.SMITH	0780789681
4	SN003	Nurse	G.OLESENG	0748596321
5	SD002	Doctor	DR.L.SMITH	0147741025
6	SA001	Administrator	N.MOKOENA	3698520147
7	SA002	Administrator	K.NAUME	3217896691
8	SR001	Receptionist	L.HOLLY	9632587410

SQL (Query Results From Oracle LiveSql) (screenshot)

APPOINTMENTS

	APPOINTMENT_ID	PATIENT_ID	APPOINTMENT_DA	APPOINTMENT_TI	APPOINTMENT_ST
1	A001	P001	4/1/2026, 12:00:00	08:00	Completed
2	A002	P002	4/3/2026, 12:00:00	09:30	Completed
3	A003	P003	4/5/2026, 12:00:00	10:00	Completed
4	A004	P004	4/8/2026, 12:00:00	11:00	Scheduled
5	A005	P005	4/9/2026, 12:00:00	13:00	Completed
6	A006	P006	4/10/2026, 12:00:0(	14:30	Scheduled
7	A007	P007	4/11/2026, 12:00:0(	15:00	Cancelled
8	A008	P008	4/12/2026, 12:00:0(	09:00	Completed

SQL (Query Results From Oracle LiveSql) (screenshot)

TREATMENTS

	TREATMENT_ID	APPOINTMENT_ID	STAFF_ID	DIAGNOSIS	TREATMENT_DESCRIPTION
1	T001	A001	SD001	Hypertension	Blood pressure treatment and monitoring
2	T004	A003	SD001	Diabetes	Blood glucose assessment

SQL (Query Results From Oracle LiveSql) (screenshot)

ADMISSIONS

	ADMISSION_ID	APPOINTMENT_ID	PATIENT_ID	ROOM_ID	ADMISSION_DATE	DISCHARGE_DATE
1	AD001	A001	P001	R01	4/1/2026, 12:00:00	4/5/2026, 12:00:00
2	AD002	A002	P002	R02	4/3/2026, 12:00:00	4/7/2026, 12:00:00
3	AD003	A005	P005	R04	4/9/2026, 12:00:00	4/12/2026, 12:00:0(

SQL (Query Results From Oracle LiveSql) (screenshot)

PRESCRIPTIONS

	PRESCRIPTION_ID	APPOINTMENT_ID	MEDICINE_ID	DOSAGE
1	PR001	A001	M001	5mg Daily
2	PR002	A001	M002	100mg Daily
3	PR003	A002	M003	500mg Every 6 Hour
4	PR004	A003	M004	500mg Twice Daily
5	PR005	A005	M002	75mg Daily
6	PR006	A008	M005	500mg Three Times

SQL (Query Results From Oracle LiveSql) (screenshot)

WARD

	WARD_ID	WARD_NAME
1	W01	General
2	W02	Surgical
3	W03	ICU

SQL (Query Results From Oracle LiveSql) (screenshot)

ROOMS

	ROOM_ID	ROOM_NUMBER	WARD_ID	ROOM_AVAILABILI
1	R01	101	W01	Occupied
2	R02	201	W02	Occupied
3	R03	102	W01	Available
4	R04	301	W03	Occupied
5	R05	202	W02	Available

SQL (Query Results From Oracle LiveSql) (screenshot)

MEDICINES

	MEDICINE_ID	MEDICINE_NAME	STOCK
1	M006	Panado	50
2	M001	Amlodipine	150
3	M002	Aspirin	300
4	M003	Paracetamol	500
5	M004	Metformin	250
6	M005	Amoxicillin	200
7	M007	Grandpa	10
8	M008	Benelin	80

SQL (Query Results From Oracle LiveSql) (screenshot)

BILLINGS

	BILL_ID	PATIENT_ID	AMOUNT	PAYMENT_STATUS
1	B001	P001	1500	Paid
2	B002	P002	2200	Pending
3	B003	P003	1800	Paid
4	B004	P004	750	Pending
5	B005	P005	3200	Paid
6	B006	P006	600	Pending
7	B007	P008	1250	Overdue
8	B008	P009	950	Pending

SQL (Sample Outcomes) (screenshot)

	TOTAL_PATIENTS
1	9

Total Number of Registered Patients

SQL

```
SELECT COUNT(*) AS total_patients
FROM PATIENTS;
```

SQL (Sample Outcomes)
(screenshot)

Available Rooms

SQL

```
SELECT room_id, room_number
FROM ROOMS
WHERE room_availability = 'Available';
```

	ROOM_ID	ROOM_NUMBER
1	R03	102
2	R05	202

	WARD_ID	WARD_NAME
1	W01	General
2	W02	Surgical
3	W03	ICU

SQL (Sample Outcomes) (screenshot)

Number of Admissions per Ward

SQL

```
SELECT w.ward_name,
COUNT(a.admission_id) total_admissions
FROM WARD w
JOIN ROOMS r ON w.ward_id = r.ward_id
JOIN ADMISSIONS a ON r.room_id = a.room_id
GROUP BY w.ward_name;
```

	PATIENT_ID	FIRST_NAME	LAST_NAME	ROOM_ID
1	P001	JOHN	DOE	R01
2	P002	JANE	DOE	R02
3	P005	WILL	FOSTER	R04

SQL (Sample Outcomes) (screenshot)

Patients Currently Admitted

SQL

```
SELECT p.patient_id,
p.first_name,
p.last_name,
a.room_id
FROM PATIENTS p
JOIN ADMISSIONS a
ON p.patient_id = a.patient_id;
```

SQL (Sample Outcomes) (screenshot)

Number of Admissions per Ward

SQL

```
SELECT w.ward_name,
COUNT(a.admission_id) total_admissions
FROM WARD w
JOIN ROOMS r ON w.ward_id = r.ward_id
JOIN ADMISSIONS a ON r.room_id = a.room_id
```

- GROUP BY w.ward_name;

	WARD_NAME	TOTAL_ADMISSIONS
1	General	1
2	Surgical	1
3	ICU	1

SQL (Sample Outcomes) (screenshot)

Treatments Performed by Each Staff Member

SQL

```
SELECT s.staff_id,
s.initials,
COUNT(t.treatment_id) treatments_done
FROM STAFF s
LEFT JOIN TREATMENTS t
ON s.staff_id = t.staff_id
GROUP BY s.staff_id,s.initials;
```

SQL (Sample Outcomes) (screenshot)

Treatments Performed by Each Staff Member (outcome)

	STAFF_ID	INITIALS	TREATMENTS_DONE
1	SD001	DR N NAIDOO	2
2	SN002	MN.SMITH	0
3	SR001	L.HOLLY	0
4	SD002	DR.L.SMITH	0
5	SA001	N.MOKOENA	0
6	SN001	L.MAHLANGU	0

6	SN001	L.MAHLANGU	0
7	SN003	G.OLESENG	0
8	SA002	K.NAUME	0

SQL (Sample Outcomes) (screenshot)

Most Prescribed Medicine

SQL

```
SELECT m.medicine_name,
COUNT(*) times_prescribed
FROM MEDICINES m
JOIN PRESCRIPTIONS p
ON m.medicine_id = p.medicine_id
GROUP BY m.medicine_name
ORDER BY times_prescribed DESC;
```

SQL (Sample Outcomes) (screenshot)

Most Prescribed Medicine (outcome)

	MEDICINE_NAME	TIMES_PRESCRIBED
1	Aspirin	2
2	Amoxicillin	1
3	Metformin	1
4	Amlodipine	1
5	Paracetamol	1

	MEDICINE_ID	MEDICINE_NAME	STOCK
1	M006	Panado	50
2	M007	Grandpa	10
3	M008	Benelin	80

SQL (Sample Outcomes) (screenshot)

Medicines with Low Stock(stock under 100)

SQL

```
SELECT medicine_id,
medicine_name,
stock
FROM MEDICINES
WHERE stock < 100;
```

SQL (Sample Outcomes) (screenshot)

Total Revenue Generated

SQL

```
SELECT SUM(amount) AS total_revenue
FROM BILLING
WHERE payment_status = 'Paid';
```

	TOTAL_REVENUE
1	6500

	BILL_ID	PATIENT_ID	AMOUNT	PAYMENT_STATUS
1	B007	P008	1250	Overdue

SQL (Sample Outcomes) (screenshot)

Overdue Bills

SQL

```
SELECT *
FROM BILLING
WHERE payment_status = 'Overdue';
```

	APPOINTMENT_STATUS	TOTAL
1	Completed	5
2	Scheduled	2
3	Cancelled	1

SQL (Sample Outcomes) (screenshot)

Grouping appointments by status

SQL

```
SELECT appointment_status,
COUNT(*) total
FROM APPOINTMENTS
GROUP BY appointment_status;
```

Conclusion

The G14 Healthcare Facility Database Management System was successfully developed to show that a relational database can be used instead of an inefficient flat-file record keeping system to provide a structured, reliable and scalable solution. The project found some major problems with manual record keeping data duplication, slow data retrieval, lack of data organisation and data integrity problems. The solution to these problems was to normalize the database structure and implement it.

The database was designed in Third Normal Form (3NF), ensuring that it is free from data duplication and relationships between patients, appointments, admissions, wards, rooms, staff, treatments, medicines, prescriptions and bill records are well represented. The end result of this ERD was a clear model of these relationships, which was useful for both outpatient and inpatient healthcare processes.

Conclusion (cont)

The implementation phase showed that the database could store, managing and retrieving healthcare information through SQL queries. They got some sample results of queries, and the system has been able to provide them with meaningful reports like the number of patients registered, availability of rooms, admissions to the hospital, statistics of treatment, medicine stock details, billing details, and revenue details etc. The results demonstrate that the database is capable of efficient decision making and enhances access to information, which is better than the original manual system.

The crow's-foot ERD shows the final design for the database.

Overall, the suggested database solution will offer G14 Healthcare Facility a robust system for dealing with healthcare documents in a unified, secure, and efficient method. The system's ability to enhance data integrity, minimize data redundancy, and facilitate quick access to information and reporting aids in the smooth running of administrative processes and provides a foundation for future expansion to address evolving healthcare information management requirements.